

Verdant

Learning notes from building the app — Milestone 1: Home, Recipe Detail, and Find Restaurants screens

These notes capture what was built, why each decision was made, and the core programming concepts introduced along the way — written as a reference for continuing to learn React Native, TypeScript, and Expo.

1. Project Setup & Folder Structure

The app is an Expo (React Native) project, configured to run on web first, with iOS/Android planned from the same codebase later.

Key folders

Concept: `index.ts` vs `App.tsx`

`index.ts` is the true entry point — it just registers App as the root component. `App.tsx` is the "lobby" that decides what's rendered: font loading, then the navigation container.

2. The Design System

A design system means defining visual values (colors, fonts, spacing, corner radius) **once**, in one place, and having every component reference them by name instead of hardcoding raw values. Change a token in one file, and the whole app updates.

Why this matters

What was built

```
src/theme/colors.ts • src/theme/typography.ts • src/theme/layout.ts
```

3. Reusable Components

Each component takes a **theme** prop rather than hardcoding a color — this is the single pattern that lets the same component work correctly on both the light Home screen and the dark Recipe Detail / Find Restaurants screens.

Components built

Key concept: `icon-as-prop`

QuickPickTile and InfoPill accept their icon as a React element (`icon: ReactNode`) rather than an icon-name string. This keeps the component decoupled from any specific icon library — it just renders whatever it's handed.

```
src/components/*.tsx
```

4. Home Screen

Built from local data files rather than hardcoded content, so a real backend can be swapped in later without changing the screen.

Pieces

```
src/screens/HomeScreen.tsx • src/data/recipes.ts • src/data/categories.ts
```

5. Real Images — and a Real Debugging Session

Free stock photos were sourced, verified (checked they actually loaded and showed the right content — one candidate for "lunch" turned out to show shrimp, wrong for a vegetarian app, and was swapped out), then bundled locally via `require()` so the app works offline and isn't dependent on an external host staying up.

The bug

Explore cards rendered as blurry, featureless color blocks instead of the actual photos, even though network requests confirmed the correct image bytes were loading successfully.

How it was diagnosed

Rather than guessing, the actual rendered DOM was inspected (`document.querySelectorAll('img')`). This revealed the `<img>` element had no sizing styles at all, and its wrapper was sized to the photo's *natural* pixel dimensions instead of the card's 140×140 size.

Root cause & fix

`StyleSheet.absoluteFill` (which positions via `top/left/right/bottom: 0`) wasn't giving React Native Web's Image component enough information to calculate "cover" scaling, so it fell back to the image's natural size. Fix: use `explicitWidth: '100%', height: '100%'` instead.

Why this matters

"The code looks right but the output is wrong" is one of the most common real situations in software engineering — especially in cross-platform frameworks. The transferable skill is inspecting actual runtime state (network tab, DOM, console) to narrow down where reality diverges from expectation, rather than guessing at fixes.

6. Navigation (react-navigation)

Introduced because simple `useState`-based screen-switching breaks down once you need real back/forward history, hardware back button support, and native-feeling transitions.

Mental model

Think of navigation as a stack of index cards. The **Tab Navigator** is a row of 5 tabs you jump between directly. Each tab can *also* have its own stack — tapping a recipe pushes "Recipe Detail" on top of the Home tab's stack; the back button pops it off, returning to exactly where you were, without disturbing the other tabs.

What was built

Honest placeholders

The floating "+" button and the Explore/Saved/Profile tabs all navigate to real "coming soon" screens rather than doing nothing — tapping a tab should always go somewhere.

```
src/navigation/TabNavigator.tsx • src/navigation/HomeStackNavigator.tsx
```

7. Recipe Detail Screen

Dark-themed screen reached by tapping the Today's Recommendation card. Custom header (back arrow, heart, share) built manually rather than using `react-navigation`'s default header, to match the fully custom premium design language used throughout the app.

Pieces

A second real TypeScript catch

The shared Theme type was originally defined as `typeof lightTheme` — which locks it to `lightTheme`'s exact hex literals, not "an object shaped like this, with any string values." This bug was invisible until `RecipeDetailScreen` became the first real caller passing `darkTheme` through a prop typed as `Theme`. Fixed with a mapped type: `{'{' } [Key in keyof typeof lightTheme]: string {'}' }` — same keys, widened value type.

```
src/screens/RecipeDetailScreen.tsx • src/components/InfoPill.tsx
```

8. Find Restaurants Screen

Reached via the Explore tab (matching the mockup, where Explore is shown active on this screen). Reuses almost everything already built — Card, Chip, custom header pattern — plus two new pieces.

New pieces

Known simplification

The filter chips (All / Vegan / Vegetarian / Gluten Free / Open Now) visually toggle correctly but don't yet filter the restaurant list — real filtering logic was deferred since the sample data set is too small to meaningfully demonstrate it.

src/screens/FindRestaurantsScreen.tsx • src/data/restaurants.ts •
src/components/RestaurantCard.tsx • src/components/MapPlaceholder.tsx

9. Glossary — Terms Introduced

Term	Meaning
Design tokens	Named, centralized values (colors, spacing, fonts) referenced by name instead of hardcoded.
Pure function	A function with no side effects — same input always gives the same output (e.g. getGreeting).
Lifting state up	Moving shared state to the nearest common parent component so multiple children can read/change it.
Local component state	State that lives inside one component because nothing outside needs to know about it (e.g. a card's selected item).
Union type	A TypeScript type restricted to a fixed set of exact values, e.g. 'breakfast' 'lunch' 'dinner'.
Mapped type	A TypeScript type built by transforming each property of another type, e.g. widening every value to <code>string</code> .
require() for assets	Bundles a local file (image, font) directly into the app at build time, instead of fetching it from a URL.
Stack navigator	Manages a linear history of screens (push/back) within one part of the app.
Tab navigator	Manages switching between top-level sections of the app, each of which may have its own stack.
Route params	Data passed into a screen via navigation (e.g. a recipe's id), instead of duplicating the full object.

10. What's Next

Generated as a learning reference for the Verdant project — Milestone 1.